

Componente de Login – parte 2



A continuación, vamos a ver los cambios necesarios para que el código funcione correctamente con la última versión de Angular y Firestore.

Código Actualizado login.component.ts:

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { AlertMessagesService } from 'jjwins-angular-alert-messages';
import { LoginService } from '../servicios/login.service';

@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
export class LoginComponent {

  email: string;
  password: string;

  constructor(
    private router: Router,
    private alertMessageService: AlertMessagesService,
    private loginService: LoginService
  ) {}
}
```

```

    ) { }

    login(){
      this.loginService.login(this.email, this.password)
        .then( () => {
          this.router.navigate(['/']);
        })
        .catch(error =>{
          this.alertMessageService.show(error.message,
            {cssClass: 'alerts-error', timeout: 4000}
          );
        });
    }
  }
}

```

Código Actualizado login.service.ts:

```

import { Injectable } from '@angular/core';
import { AngularFireAuth } from '@angular/fire/compat/auth';

@Injectable()
export class LoginService {
  constructor(private authService: AngularFireAuth) {}

  login(email: string, password: string) {
    return new Promise((resolve, reject) => {
      this.authService.signInWithEmailAndPassword(email, password)
        .then(datos => resolve(datos))
        .catch(error => reject(error));
    });
  }
}

```

Aquí te detallo las diferencias entre el código "anterior" y el "nuevo" código para el método login, junto con las razones de los cambios realizados.

Diferencias y Explicaciones

1. Acceso Directo a signInWithEmailAndPassword

- **Anterior:**

```
this.authService.auth.signInWithEmailAndPassword(email, password)
```

- **Nuevo:**

```
this.authService.signInWithEmailAndPassword(email, password)
```

Razón:

- **Cambio en la API de Firebase:**
 - En versiones más recientes de @angular/fire, la propiedad auth ha sido eliminada. Ahora, los métodos de autenticación, como signInWithEmailAndPassword, se acceden directamente desde la instancia de AngularFireAuth (this.authService). Este cambio simplifica el acceso a los métodos y reduce la profundidad de la propiedad.

2. Cambio en el Manejo de Promesas

- **Anterior:**

```
.then(datos => resolve(datos),  
      error => reject(error)  
)
```

- **Nuevo:**

```
.then(datos => resolve(datos))  
.catch(error => reject(error));
```

Razón:

- **Separación de then y catch:**
 - En el código viejito, tanto la resolución (resolve) como el rechazo (reject) de la promesa estaban manejados en un solo bloque then, lo que puede ser menos claro y propenso a errores en algunos casos.
 - El código nuevo utiliza un enfoque más estándar y moderno, separando la lógica de éxito (then) de la lógica de error (catch). Esto mejora la claridad y mantiene la promesa más fácil de entender y mantener.

3. Eliminación de los Tipos Implícitos en datos y error

- **Anterior:**

```
.then(datos => resolve(datos),  
      error => reject(error)  
)
```

- **Nuevo:**

```
.then(datos => resolve(datos))  
.catch(error => reject(error));
```

Razón:

- **Mejoras en el Tipado de TypeScript:**
 - Aunque en el código proporcionado no se especifican explícitamente los tipos, el cambio a un `catch` separado permite a TypeScript inferir mejor los tipos de `datos` y `error`. Si fuera necesario, podrías definir explícitamente los tipos para `datos` y `error`, pero el uso del `catch` separado mejora la claridad y la gestión de errores.

Resumen de los Cambios

1. **Acceso Directo a Métodos de `AngularFireAuth`:** En el código nuevo, se elimina el acceso a través de `auth`, simplificando el acceso a los métodos de autenticación.
2. **Manejo Mejorado de Promesas:** Separar `then` y `catch` es una práctica estándar que mejora la claridad y la robustez del código, asegurando una mejor gestión de errores.
3. **Mejora en el Tipado:** Aunque no se especifican los tipos, la estructura nueva permite una mejor inferencia de tipos por parte de TypeScript, lo que mejora la seguridad y la mantenibilidad del código.

Estos cambios reflejan una actualización hacia prácticas más modernas y mejores estándares en la gestión de autenticación y manejo de promesas en aplicaciones Angular utilizando Firebase.

Recuerden que pueden descargar el código actualizado en la sección de recursos de esta lección.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](http://www.globalmentoring.com.mx)